Chapter 3: Numbers, Dates, and Times

Time Offsets, ISO Conversion, Date Ranges, and Timezone Arithmetic

Source: Python Cookbook, 3rd Edition (David Beazley & Brian K. Jones) | Study Edition for Autonomous Calendar Scheduler (Week 1)

Table of Contents & Core Learning Outline

• **Recipe 3.12: Time Conversions** — datetime.timedelta calculations between days, hours, and seconds

• **Recipe 3.13: Determining Date Offsets** — Calculating dynamic dates (e.g. next Friday, tomorrow)

• **Recipe 3.14: Finding Date Ranges** — Generating step intervals and windows for schedules

• **Recipe 3.15: String to Datetime Parsing** — Converting ISO 8601 strings to datetime objects and vice versa

• **Recipe 3.16: Timezone Manipulation** — Working with UTC, local offsets (pytz / zoneinfo), and Daylight Saving

---

Print & Study Note: *This document contains the full unabridged chapter text following this Table of Contents page. Focus on understanding the core design patterns and self-implementing the exercises without copy-pasting AI code.*

CHAPTER 3

Numbers, Dates, and Times

Performing mathematical calculations with integers and floating-point numbers is easy in Python. However, if you need to perform calculations with fractions, arrays, or dates and times, a bit more work is required. The focus of this chapter is on such topics.

3.1. Rounding Numerical Values

Problem

You want to round a floating-point number to a fixed number of decimal places.

Solution

For simple rounding, use the built-in `round(value, ndigits)` function. For example:

```
>>> round(1.23, 1)
1.2
>>> round(1.27, 1)
1.3
>>> round(-1.27, 1)
-1.3
>>> round(1.25361,3)
1.254
>>>
```

When a value is exactly halfway between two choices, the behavior of round is to round to the nearest even digit. That is, values such as 1.5 or 2.5 both get rounded to 2.

The number of digits given to `round()` can be negative, in which case rounding takes place for tens, hundreds, thousands, and so on. For example:

```
>>> a = 1627731
>>> round(a, -1)
1627730
```

```
>>> round(a, -2)
1627700
>>> round(a, -3)
1628000
>>>
```

Discussion

Don't confuse rounding with formatting a value for output. If your goal is simply to output a numerical value with a certain number of decimal places, you don't typically need to use `round()`. Instead, just specify the desired precision when formatting. For example:

```
>>> x = 1.23456
>>> format(x, '0.2f')
'1.23'
>>> format(x, '0.3f')
'1.235'
>>> 'value is {:0.3f}'.format(x)
'value is 1.235'
>>>
```

Also, resist the urge to round floating-point numbers to "fix" perceived accuracy problems. For example, you might be inclined to do this:

```
>>> a = 2.1
>>> b = 4.2
>>> c = a + b
>>> c
6.300000000000001
>>> c = round(c, 2)      # "Fix" result (???)
>>> c
6.3
>>>
```

For most applications involving floating point, it's simply not necessary (or recommended) to do this. Although there are small errors introduced into calculations, the behavior of those errors are understood and tolerated. If avoiding such errors is important (e.g., in financial applications, perhaps), consider the use of the `decimal` module, which is discussed in the next recipe.

3.2. Performing Accurate Decimal Calculations

Problem

You need to perform accurate calculations with decimal numbers, and don't want the small errors that naturally occur with floats.

Solution

A well-known issue with floating-point numbers is that they can't accurately represent all base-10 decimals. Moreover, even simple mathematical calculations introduce small errors. For example:

```
>>> a = 4.2
>>> b = 2.1
>>> a + b
6.300000000000001
>>> (a + b) == 6.3
False
>>>
```

These errors are a "feature" of the underlying CPU and the IEEE 754 arithmetic performed by its floating-point unit. Since Python's float data type stores data using the native representation, there's nothing you can do to avoid such errors if you write your code using `float` instances.

If you want more accuracy (and are willing to give up some performance), you can use the `decimal` module:

```
>>> from decimal import Decimal
>>> a = Decimal('4.2')
>>> b = Decimal('2.1')
>>> a + b
Decimal('6.3')
>>> print(a + b)
6.3
>>> (a + b) == Decimal('6.3')
True
>>>
```

At first glance, it might look a little weird (i.e., specifying numbers as strings). However, `Decimal` objects work in every way that you would expect them to (supporting all of the usual math operations, etc.). If you print them or use them in string formatting functions, they look like normal numbers.

A major feature of `decimal` is that it allows you to control different aspects of calculations, including number of digits and rounding. To do this, you create a local context and change its settings. For example:

```
>>> from decimal import localcontext
>>> a = Decimal('1.3')
>>> b = Decimal('1.7')
>>> print(a / b)
0.7647058823529411764705882353
>>> with localcontext() as ctx:
...     ctx.prec = 3
...     print(a / b)
...
```

```
0.765
>>> with localcontext() as ctx:
...     ctx.prec = 50
...     print(a / b)
...
0.76470588235294117647058823529411764705882352941176
>>>
```

Discussion

The `decimal` module implements IBM's "General Decimal Arithmetic Specification." Needless to say, there are a huge number of configuration options that are beyond the scope of this book.

Newcomers to Python might be inclined to use the `decimal` module to work around perceived accuracy problems with the `float` data type. However, it's really important to understand your application domain. If you're working with science or engineering problems, computer graphics, or most things of a scientific nature, it's simply more common to use the normal floating-point type. For one, very few things in the real world are measured to the 17 digits of accuracy that floats provide. Thus, tiny errors introduced in calculations just don't matter. Second, the performance of native floats is significantly faster—something that's important if you're performing a large number of calculations.

That said, you can't ignore the errors completely. Mathematicians have spent a lot of time studying various algorithms, and some handle errors better than others. You also have to be a little careful with effects due to things such as subtractive cancellation and adding large and small numbers together. For example:

```
>>> nums = [1.23e+18, 1, -1.23e+18]
>>> sum(nums)    # Notice how 1 disappears
0.0
>>>
```

This latter example can be addressed by using a more accurate implementation in `math.fsum()`:

```
>>> import math
>>> math.fsum(nums)
1.0
>>>
```

However, for other algorithms, you really need to study the algorithm and understand its error propagation properties.

All of this said, the main use of the `decimal` module is in programs involving things such as finance. In such programs, it is extremely annoying to have small errors creep into the calculation. Thus, `decimal` provides a way to avoid that. It is also common to encounter `Decimal` objects when Python interfaces with databases—again, especially when accessing financial data.

3.3. Formatting Numbers for Output

Problem

You need to format a number for output, controlling the number of digits, alignment, inclusion of a thousands separator, and other details.

Solution

To format a single number for output, use the built-in `format()` function. For example:

```
>>> x = 1234.56789

>>> # Two decimal places of accuracy
>>> format(x, '0.2f')
'1234.57'

>>> # Right justified in 10 chars, one-digit accuracy
>>> format(x, '>10.1f')
'    1234.6'

>>> # Left justified
>>> format(x, '<10.1f')
'1234.6    '

>>> # Centered
>>> format(x, '^10.1f')
'  1234.6  '

>>> # Inclusion of thousands separator
>>> format(x, ',')
'1,234.56789'
>>> format(x, '0,.1f')
'1,234.6'
>>>
```

If you want to use exponential notation, change the `f` to an `e` or `E`, depending on the case you want used for the exponential specifier. For example:

```
>>> format(x, 'e')
'1.234568e+03'
>>> format(x, '0.2E')
'1.23E+03'
>>>
```

The general form of the width and precision in both cases is `'[<>^]?width[,]?(.digits)?'` where `width` and `digits` are integers and `?` signifies optional parts. The same format codes are also used in the `.format()` method of strings. For example:

```
>>> 'The value is {:0,.2f}'.format(x)
'The value is 1,234.57'
>>>
```

Discussion

Formatting numbers for output is usually straightforward. The technique shown works for both floating-point numbers and `Decimal` numbers in the `decimal` module.

When the number of digits is restricted, values are rounded away according to the same rules of the `round()` function. For example:

```
>>> x
1234.56789
>>> format(x, '0.1f')
'1234.6'
>>> format(-x, '0.1f')
'-1234.6'
>>>
```

Formatting of values with a thousands separator is not locale aware. If you need to take that into account, you might investigate functions in the `locale` module. You can also swap separator characters using the `translate()` method of strings. For example:

```
>>> swap_separators = { ord('.'):',', ord(','):'.' }
>>> format(x, ',').translate(swap_separators)
'1.234,56789'
>>>
```

In a lot of Python code, numbers are formatted using the `%` operator. For example:

```
>>> '%0.2f' % x
'1234.57'
>>> '%10.1f' % x
'    1234.6'
>>> '%-10.1f' % x
'1234.6    '
>>>
```

This formatting is still acceptable, but less powerful than the more modern `format()` method. For example, some features (e.g., adding thousands separators) aren't supported when using the `%` operator to format numbers.

3.4. Working with Binary, Octal, and Hexadecimal Integers

Problem

You need to convert or output integers represented by binary, octal, or hexadecimal digits.

Solution

To convert an integer into a binary, octal, or hexadecimal text string, use the `bin()`, `oct()`, or `hex()` functions, respectively:

```
>>> x = 1234
>>> bin(x)
'0b10011010010'
>>> oct(x)
'0o2322'
>>> hex(x)
'0x4d2'
>>>
```

Alternatively, you can use the `format()` function if you don't want the `0b`, `0o`, or `0x` prefixes to appear. For example:

```
>>> format(x, 'b')
'10011010010'
>>> format(x, 'o')
'2322'
>>> format(x, 'x')
'4d2'
>>>
```

Integers are signed, so if you are working with negative numbers, the output will also include a sign. For example:

```
>>> x = -1234
>>> format(x, 'b')
'-10011010010'
>>> format(x, 'x')
'-4d2'
>>>
```

If you need to produce an unsigned value instead, you'll need to add in the maximum value to set the bit length. For example, to show a 32-bit value, use the following:

```
>>> x = -1234
>>> format(2**32 + x, 'b')
'11111111111111111111101100101110'
>>> format(2**32 + x, 'x')
```

```
'fffffb2e'
>>>
```

To convert integer strings in different bases, simply use the `int()` function with an appropriate base. For example:

```
>>> int('4d2', 16)
1234
>>> int('10011010010', 2)
1234
>>>
```

Discussion

For the most part, working with binary, octal, and hexadecimal integers is straightforward. Just remember that these conversions only pertain to the conversion of integers to and from a textual representation. Under the covers, there's just one integer type.

Finally, there is one caution for programmers who use octal. The Python syntax for specifying octal values is slightly different than many other languages. For example, if you try something like this, you'll get a syntax error:

```
>>> import os
>>> os.chmod('script.py', 0755)
  File "<stdin>", line 1
    os.chmod('script.py', 0755)
                          ^
SyntaxError: invalid token
>>>
```

Make sure you prefix the octal value with `0o`, as shown here:

```
>>> os.chmod('script.py', 0o755)
>>>
```

3.5. Packing and Unpacking Large Integers from Bytes

Problem

You have a byte string and you need to unpack it into an integer value. Alternatively, you need to convert a large integer back into a byte string.

Solution

Suppose your program needs to work with a 16-element byte string that holds a 128-bit integer value. For example:

```
data = b'\x00\x124V\x00x\x90\xab\x00\xcd\xef\x01\x00#\x004'
```

To interpret the bytes as an integer, use `int.from_bytes()`, and specify the byte ordering like this:

```
>>> len(data)
16
>>> int.from_bytes(data, 'little')
69120565665751139577663547927094891008
>>> int.from_bytes(data, 'big')
94522842520747284487117727783387188
>>>
```

To convert a large integer value back into a byte string, use the `int.to_bytes()` method, specifying the number of bytes and the byte order. For example:

```
>>> x = 94522842520747284487117727783387188
>>> x.to_bytes(16, 'big')
b'\x00\x124V\x00x\x90\xab\x00\xcd\xef\x01\x00#\x004'
>>> x.to_bytes(16, 'little')
b'4\x00#\x00\x01\xef\xcd\x00\xab\x90x\x00V4\x12\x00'
>>>
```

Discussion

Converting large integer values to and from byte strings is not a common operation. However, it sometimes arises in certain application domains, such as cryptography or networking. For instance, IPv6 network addresses are represented as 128-bit integers. If you are writing code that needs to pull such values out of a data record, you might face this problem.

As an alternative to this recipe, you might be inclined to unpack values using the `struct` module, as described in Recipe 6.11. This works, but the size of integers that can be unpacked with `struct` is limited. Thus, you would need to unpack multiple values and combine them to create the final value. For example:

```
>>> data
b'\x00\x124V\x00x\x90\xab\x00\xcd\xef\x01\x00#\x004'
>>> import struct
>>> hi, lo = struct.unpack('>QQ', data)
>>> (hi << 64) + lo
94522842520747284487117727783387188
>>>
```

The specification of the byte order (`little` or `big`) just indicates whether the bytes that make up the integer value are listed from the least to most significant or the other way around. This is easy to view using a carefully crafted hexadecimal value:

```
>>> x = 0x01020304
>>> x.to_bytes(4, 'big')
b'\x01\x02\x03\x04'
>>> x.to_bytes(4, 'little')
```

```
b'\x04\x03\x02\x01'
>>>
```

If you try to pack an integer into a byte string, but it won't fit, you'll get an error. You can use the `int.bit_length()` method to determine how many bits are required to store a value if needed:

```
>>> x = 523 ** 23
>>> x
335381300113661875107536852714019056160355655333978849017944067
>>> x.to_bytes(16, 'little')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: int too big to convert
>>> x.bit_length()
208
>>> nbytes, rem = divmod(x.bit_length(), 8)
>>> if rem:
...     nbytes += 1
...
>>>
>>> x.to_bytes(nbytes, 'little')
b'\x03X\xf1\x82iT\x96\xac\xc7c\x16\xf3\xb9\xcf...\xd0'
>>>
```

3.6. Performing Complex-Valued Math

Problem

Your code for interacting with the latest web authentication scheme has encountered a singularity and your only solution is to go around it in the complex plane. Or maybe you just need to perform some calculations using complex numbers.

Solution

Complex numbers can be specified using the `complex(real, imag)` function or by floating-point numbers with a `j` suffix. For example:

```
>>> a = complex(2, 4)
>>> b = 3 - 5j
>>> a
(2+4j)
>>> b
(3-5j)
>>>
```

The real, imaginary, and conjugate values are easy to obtain, as shown here:

```
>>> a.real
2.0
```

```
>>> a.imag
4.0
>>> a.conjugate()
(2-4j)
>>>
```

In addition, all of the usual mathematical operators work:

```
>>> a + b
(5-1j)
>>> a * b
(26+2j)
>>> a / b
(-0.411764705882353529+0.6470588235294118j)
>>> abs(a)
4.47213595499958
>>>
```

To perform additional complex-valued functions such as sines, cosines, or square roots, use the `cmath` module:

```
>>> import cmath
>>> cmath.sin(a)
(24.83130584894638-11.356612711218174j)
>>> cmath.cos(a)
(-11.36423470640106-24.814651485634187j)
>>> cmath.exp(a)
(-4.829809383269385-5.5920560936409816j)
>>>
```

Discussion

Most of Python's math-related modules are aware of complex values. For example, if you use `numpy`, it is straightforward to make arrays of complex values and perform operations on them:

```
>>> import numpy as np
>>> a = np.array([2+3j, 4+5j, 6-7j, 8+9j])
>>> a
array([ 2.+3.j,  4.+5.j,  6.-7.j,  8.+9.j])
>>> a + 2
array([  4.+3.j,   6.+5.j,   8.-7.j,  10.+9.j])
>>> np.sin(a)
array([    9.15449915  -4.16890696j,   -56.16227422 -48.50245524j,
        -153.20827755-526.47684926j,  4008.42651446-589.49948373j])
>>>
```

Python's standard mathematical functions do not produce complex values by default, so it is unlikely that such a value would accidentally show up in your code. For example:

```
>>> import math
>>> math.sqrt(-1)
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
ValueError: math domain error
>>>
```

If you want complex numbers to be produced as a result, you have to explicitly use `cmath` or declare the use of a complex type in libraries that know about them. For example:

```
>>> import cmath
>>> cmath.sqrt(-1)
1j
>>>
```

3.7. Working with Infinity and NaNs

Problem

You need to create or test for the floating-point values of infinity, negative infinity, or NaN (not a number).

Solution

Python has no special syntax to represent these special floating-point values, but they can be created using `float()`. For example:

```
>>> a = float('inf')
>>> b = float('-inf')
>>> c = float('nan')
>>> a
inf
>>> b
-inf
>>> c
nan
>>>
```

To test for the presence of these values, use the `math.isinf()` and `math.isnan()` functions. For example:

```
>>> math.isinf(a)
True
>>> math.isnan(c)
True
>>>
```

Discussion

For more detailed information about these special floating-point values, you should refer to the IEEE 754 specification. However, there are a few tricky details to be aware of, especially related to comparisons and operators.

Infinite values will propagate in calculations in a mathematical manner. For example:

```
>>> a = float('inf')
>>> a + 45
inf
>>> a * 10
inf
>>> 10 / a
0.0
>>>
```

However, certain operations are undefined and will result in a NaN result. For example:

```
>>> a = float('inf')
>>> a/a
nan
>>> b = float('-inf')
>>> a + b
nan
>>>
```

NaN values propagate through all operations without raising an exception. For example:

```
>>> c = float('nan')
>>> c + 23
nan
>>> c / 2
nan
>>> c * 2
nan
>>> math.sqrt(c)
nan
>>>
```

A subtle feature of NaN values is that they never compare as equal. For example:

```
>>> c = float('nan')
>>> d = float('nan')
>>> c == d
False
>>> c is d
False
>>>
```

Because of this, the only safe way to test for a NaN value is to use `math.isnan()`, as shown in this recipe.

Sometimes programmers want to change Python's behavior to raise exceptions when operations result in an infinite or NaN result. The `fpectl` module can be used to adjust this behavior, but it is not enabled in a standard Python build, it's platform-dependent, and really only intended for expert-level programmers. See the online Python documentation for further details.

3.8. Calculating with Fractions

Problem

You have entered a time machine and suddenly find yourself working on elementary-level homework problems involving fractions. Or perhaps you're writing code to make calculations involving measurements made in your wood shop.

Solution

The `fractions` module can be used to perform mathematical calculations involving fractions. For example:

```
>>> from fractions import Fraction
>>> a = Fraction(5, 4)
>>> b = Fraction(7, 16)
>>> print(a + b)
27/16
>>> print(a * b)
35/64

>>> # Getting numerator/denominator
>>> c = a * b
>>> c.numerator
35
>>> c.denominator
64

>>> # Converting to a float
>>> float(c)
0.546875

>>> # Limiting the denominator of a value
>>> print(c.limit_denominator(8))
4/7

>>> # Converting a float to a fraction
>>> x = 3.75
>>> y = Fraction(*x.as_integer_ratio())
>>> y
Fraction(15, 4)
>>>
```

Discussion

Calculating with fractions doesn't arise often in most programs, but there are situations where it might make sense to use them. For example, allowing a program to accept units of measurement in fractions and performing calculations with them in that form might alleviate the need for a user to manually make conversions to decimals or floats.

3.9. Calculating with Large Numerical Arrays

Problem

You need to perform calculations on large numerical datasets, such as arrays or grids.

Solution

For any heavy computation involving arrays, use the NumPy library. The major feature of NumPy is that it gives Python an array object that is much more efficient and better suited for mathematical calculation than a standard Python list. Here is a short example illustrating important behavioral differences between lists and NumPy arrays:

```
>>> # Python lists
>>> x = [1, 2, 3, 4]
>>> y = [5, 6, 7, 8]
>>> x * 2
[1, 2, 3, 4, 1, 2, 3, 4]
>>> x + 10
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate list (not "int") to list
>>> x + y
[1, 2, 3, 4, 5, 6, 7, 8]

>>> # Numpy arrays
>>> import numpy as np
>>> ax = np.array([1, 2, 3, 4])
>>> ay = np.array([5, 6, 7, 8])
>>> ax * 2
array([2, 4, 6, 8])
>>> ax + 10
array([11, 12, 13, 14])
>>> ax + ay
array([ 6,  8, 10, 12])
>>> ax * ay
array([ 5, 12, 21, 32])
>>>
```

As you can see, basic mathematical operations involving arrays behave differently. Specifically, scalar operations (e.g., `ax * 2` or `ax + 10`) apply the operation on an element-by-element basis. In addition, performing math operations when both operands are arrays applies the operation to all elements and produces a new array.

The fact that math operations apply to all of the elements simultaneously makes it very easy and fast to compute functions across an entire array. For example, if you want to compute the value of a polynomial:

```
>>> def f(x):
...     return 3*x**2 - 2*x + 7
```

```
...
>>> f(ax)
array([ 8, 15, 28, 47])
>>>
```

NumPy provides a collection of "universal functions" that also allow for array operations. These are replacements for similar functions normally found in the `math` module. For example:

```
>>> np.sqrt(ax)
array([ 1.        ,  1.41421356,  1.73205081,  2.        ])
>>> np.cos(ax)
array([ 0.54030231, -0.41614684, -0.9899925 , -0.65364362])
>>>
```

Using universal functions can be hundreds of times faster than looping over the array elements one at a time and performing calculations using functions in the `math` module. Thus, you should prefer their use whenever possible.

Under the covers, NumPy arrays are allocated in the same manner as in C or Fortran. Namely, they are large, contiguous memory regions consisting of a homogenous data type. Because of this, it's possible to make arrays much larger than anything you would normally put into a Python list. For example, if you want to make a two-dimensional grid of 10,000 by 10,000 floats, it's not an issue:

```
>>> grid = np.zeros(shape=(10000,10000), dtype=float)
>>> grid
array([[ 0.,  0.,  0., ...,  0.,  0.,  0.],
       [ 0.,  0.,  0., ...,  0.,  0.,  0.],
       [ 0.,  0.,  0., ...,  0.,  0.,  0.],
       ...,
       [ 0.,  0.,  0., ...,  0.,  0.,  0.],
       [ 0.,  0.,  0., ...,  0.,  0.,  0.],
       [ 0.,  0.,  0., ...,  0.,  0.,  0.]])
>>>
```

All of the usual operations still apply to all of the elements simultaneously:

```
>>> grid += 10
>>> grid
array([[ 10.,  10.,  10., ...,  10.,  10.,  10.],
       [ 10.,  10.,  10., ...,  10.,  10.,  10.],
       [ 10.,  10.,  10., ...,  10.,  10.,  10.],
       ...,
       [ 10.,  10.,  10., ...,  10.,  10.,  10.],
       [ 10.,  10.,  10., ...,  10.,  10.,  10.],
       [ 10.,  10.,  10., ...,  10.,  10.,  10.]])
>>> np.sin(grid)
array([[-0.54402111, -0.54402111, -0.54402111, ..., -0.54402111,
        -0.54402111, -0.54402111],
       [-0.54402111, -0.54402111, -0.54402111, ..., -0.54402111,
        -0.54402111, -0.54402111],
```

```
       [-0.54402111, -0.54402111, -0.54402111, ..., -0.54402111,
        -0.54402111, -0.54402111],
       ...,
       [-0.54402111, -0.54402111, -0.54402111, ..., -0.54402111,
        -0.54402111, -0.54402111],
       [-0.54402111, -0.54402111, -0.54402111, ..., -0.54402111,
        -0.54402111, -0.54402111],
       [-0.54402111, -0.54402111, -0.54402111, ..., -0.54402111,
        -0.54402111, -0.54402111]])
>>>
```

One extremely notable aspect of NumPy is the manner in which it extends Python's list indexing functionality—especially with multidimensional arrays. To illustrate, make a simple two-dimensional array and try some experiments:

```
>>> a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
>>> a
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])

>>> # Select row 1
>>> a[1]
array([5, 6, 7, 8])

>>> # Select column 1
>>> a[:,1]
array([ 2,  6, 10])

>>> # Select a subregion and change it
>>> a[1:3, 1:3]
array([[ 6,  7],
       [10, 11]])
>>> a[1:3, 1:3] += 10
>>> a
array([[ 1,  2,  3,  4],
       [ 5, 16, 17,  8],
       [ 9, 20, 21, 12]])

>>> # Broadcast a row vector across an operation on all rows
>>> a + [100, 101, 102, 103]
array([[101, 103, 105, 107],
       [105, 117, 119, 111],
       [109, 121, 123, 115]])
>>> a
array([[ 1,  2,  3,  4],
       [ 5, 16, 17,  8],
       [ 9, 20, 21, 12]])

>>> # Conditional assignment on an array
>>> np.where(a < 10, a, 10)
array([[ 1,  2,  3,  4],
```

```
        [ 5, 10, 10,  8],
        [ 9, 10, 10, 10]])
>>>
```

Discussion

NumPy is the foundation for a huge number of science and engineering libraries in Python. It is also one of the largest and most complicated modules in widespread use. That said, it's still possible to accomplish useful things with NumPy by starting with simple examples and playing around.

One note about usage is that it is relatively common to use the statement `import numpy as np`, as shown in the solution. This simply shortens the name to something that's more convenient to type over and over again in your program.

For more information, you definitely need to visit *http://www.numpy.org*.

3.10. Performing Matrix and Linear Algebra Calculations

Problem

You need to perform matrix and linear algebra operations, such as matrix multiplication, finding determinants, solving linear equations, and so on.

Solution

The NumPy library has a `matrix` object that can be used for this purpose. Matrices are somewhat similar to the array objects described in Recipe 3.9, but follow linear algebra rules for computation. Here is an example that illustrates a few essential features:

```
>>> import numpy as np
>>> m = np.matrix([[1,-2,3],[0,4,5],[7,8,-9]])
>>> m
matrix([[ 1, -2,  3],
        [ 0,  4,  5],
        [ 7,  8, -9]])

>>> # Return transpose
>>> m.T
matrix([[ 1,  0,  7],
        [-2,  4,  8],
        [ 3,  5, -9]])

>>> # Return inverse
>>> m.I
matrix([[ 0.33043478, -0.02608696,  0.09565217],
        [-0.15217391,  0.13043478,  0.02173913],
        [ 0.12173913,  0.09565217, -0.0173913 ]])
```

```
>>> # Create a vector and multiply
>>> v = np.matrix([[2],[3],[4]])
>>> v
matrix([[2],
        [3],
        [4]])
>>> m * v
matrix([[ 8],
        [32],
        [ 2]])
>>>
```

More operations can be found in the `numpy.linalg` subpackage. For example:

```
>>> import numpy.linalg

>>> # Determinant
>>> numpy.linalg.det(m)
-229.99999999999983

>>> # Eigenvalues
>>> numpy.linalg.eigvals(m)
array([-13.11474312,   2.75956154,   6.35518158])

>>> # Solve for x in mx = v
>>> x = numpy.linalg.solve(m, v)
>>> x
matrix([[ 0.96521739],
        [ 0.17391304],
        [ 0.46086957]])
>>> m * x
matrix([[ 2.],
        [ 3.],
        [ 4.]])
>>> v
matrix([[2],
        [3],
        [4]])
>>>
```

Discussion

Linear algebra is obviously a huge topic that's far beyond the scope of this cookbook. However, if you need to manipulate matrices and vectors, NumPy is a good starting point. Visit *http://www.numpy.org* for more detailed information.

3.11. Picking Things at Random

Problem

You want to pick random items out of a sequence or generate random numbers.

Solution

The `random` module has various functions for random numbers and picking random items. For example, to pick a random item out of a sequence, use `random.choice()`:

```
>>> import random
>>> values = [1, 2, 3, 4, 5, 6]
>>> random.choice(values)
2
>>> random.choice(values)
3
>>> random.choice(values)
1
>>> random.choice(values)
4
>>> random.choice(values)
6
>>>
```

To take a sampling of N items where selected items are removed from further consideration, use `random.sample()` instead:

```
>>> random.sample(values, 2)
[6, 2]
>>> random.sample(values, 2)
[4, 3]
>>> random.sample(values, 3)
[4, 3, 1]
>>> random.sample(values, 3)
[5, 4, 1]
>>>
```

If you simply want to shuffle items in a sequence in place, use `random.shuffle()`:

```
>>> random.shuffle(values)
>>> values
[2, 4, 6, 5, 3, 1]
>>> random.shuffle(values)
>>> values
[3, 5, 2, 1, 6, 4]
>>>
```

To produce random integers, use `random.randint()`:

```
>>> random.randint(0,10)
2
```

```
>>> random.randint(0,10)
5
>>> random.randint(0,10)
0
>>> random.randint(0,10)
7
>>> random.randint(0,10)
10
>>> random.randint(0,10)
3
>>>
```

To produce uniform floating-point values in the range 0 to 1, use `random.random()`:

```
>>> random.random()
0.9406677561675867
>>> random.random()
0.133129581343897
>>> random.random()
0.4144991136919316
>>>
```

To get N random-bits expressed as an integer, use `random.getrandbits()`:

```
>>> random.getrandbits(200)
335837000776573622800628485064121869519521710558559406913275
>>>
```

Discussion

The `random` module computes random numbers using the Mersenne Twister algorithm. This is a deterministic algorithm, but you can alter the initial seed by using the `random.seed()` function. For example:

```
random.seed()            # Seed based on system time or os.urandom()
random.seed(12345)       # Seed based on integer given
random.seed(b'bytedata') # Seed based on byte data
```

In addition to the functionality shown, `random()` includes functions for uniform, Gaussian, and other probabality distributions. For example, `random.uniform()` computes uniformly distributed numbers, and `random.gauss()` computes normally distributed numbers. Consult the documentation for information on other supported distributions.

Functions in `random()` should not be used in programs related to cryptography. If you need such functionality, consider using functions in the `ssl` module instead. For example, `ssl.RAND_bytes()` can be used to generate a cryptographically secure sequence of random bytes.

3.12. Converting Days to Seconds, and Other Basic Time Conversions

Problem

You have code that needs to perform simple time conversions, like days to seconds, hours to minutes, and so on.

Solution

To perform conversions and arithmetic involving different units of time, use the `datetime` module. For example, to represent an interval of time, create a `timedelta` instance, like this:

```
>>> from datetime import timedelta
>>> a = timedelta(days=2, hours=6)
>>> b = timedelta(hours=4.5)
>>> c = a + b
>>> c.days
2
>>> c.seconds
37800
>>> c.seconds / 3600
10.5
>>> c.total_seconds() / 3600
58.5
>>>
```

If you need to represent specific dates and times, create `datetime` instances and use the standard mathematical operations to manipulate them. For example:

```
>>> from datetime import datetime
>>> a = datetime(2012, 9, 23)
>>> print(a + timedelta(days=10))
2012-10-03 00:00:00
>>>
>>> b = datetime(2012, 12, 21)
>>> d = b - a
>>> d.days
89
>>> now = datetime.today()
>>> print(now)
2012-12-21 14:54:43.094063
>>> print(now + timedelta(minutes=10))
2012-12-21 15:04:43.094063
>>>
```

When making calculations, it should be noted that `datetime` is aware of leap years. For example:

```
>>> a = datetime(2012, 3, 1)
>>> b = datetime(2012, 2, 28)
>>> a - b
datetime.timedelta(2)
>>> (a - b).days
2
>>> c = datetime(2013, 3, 1)
>>> d = datetime(2013, 2, 28)
>>> (c - d).days
1
>>>
```

Discussion

For most basic date and time manipulation problems, the `datetime` module will suffice. If you need to perform more complex date manipulations, such as dealing with time zones, fuzzy time ranges, calculating the dates of holidays, and so forth, look at the `dateutil` module.

To illustrate, many similar time calculations can be performed with the `dateutil.relativedelta()` function. However, one notable feature is that it fills in some gaps pertaining to the handling of months (and their differing number of days). For instance:

```
>>> a = datetime(2012, 9, 23)
>>> a + timedelta(months=1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'months' is an invalid keyword argument for this function
>>>

>>> from dateutil.relativedelta import relativedelta
>>> a + relativedelta(months=+1)
datetime.datetime(2012, 10, 23, 0, 0)
>>> a + relativedelta(months=+4)
datetime.datetime(2013, 1, 23, 0, 0)
>>>

>>> # Time between two dates
>>> b = datetime(2012, 12, 21)
>>> d = b - a
>>> d
datetime.timedelta(89)
>>> d = relativedelta(b, a)
>>> d
relativedelta(months=+2, days=+28)
>>> d.months
2
>>> d.days
28
>>>
```

3.13. Determining Last Friday's Date

Problem

You want a general solution for finding a date for the last occurrence of a day of the week. Last Friday, for example.

Solution

Python's `datetime` module has utility functions and classes to help perform calculations like this. A decent, generic solution to this problem looks like this:

```
from datetime import datetime, timedelta

weekdays = ['Monday', 'Tuesday', 'Wednesday', 'Thursday',
            'Friday', 'Saturday', 'Sunday']

def get_previous_byday(dayname, start_date=None):
    if start_date is None:
        start_date = datetime.today()
    day_num = start_date.weekday()
    day_num_target = weekdays.index(dayname)
    days_ago = (7 + day_num - day_num_target) % 7
    if days_ago == 0:
        days_ago = 7
    target_date = start_date - timedelta(days=days_ago)
    return target_date
```

Using this in an interpreter session would look like this:

```
>>> datetime.today()  # For reference
datetime.datetime(2012, 8, 28, 22, 4, 30, 263076)
>>> get_previous_byday('Monday')
datetime.datetime(2012, 8, 27, 22, 3, 57, 29045)
>>> get_previous_byday('Tuesday') # Previous week, not today
datetime.datetime(2012, 8, 21, 22, 4, 12, 629771)
>>> get_previous_byday('Friday')
datetime.datetime(2012, 8, 24, 22, 5, 9, 911393)
>>>
```

The optional `start_date` can be supplied using another `datetime` instance. For example:

```
>>> get_previous_byday('Sunday', datetime(2012, 12, 21))
datetime.datetime(2012, 12, 16, 0, 0)
>>>
```

Discussion

This recipe works by mapping the start date and the target date to their numeric position in the week (with `Monday` as day 0). Modular arithmetic is then used to figure out how many days ago the target date last occurred. From there, the desired date is calculated from the start date by subtracting an appropriate `timedelta` instance.

If you're performing a lot of date calculations like this, you may be better off installing the `python-dateutil` package instead. For example, here is an example of performing the same calculation using the `relativedelta()` function from `dateutil`:

```
>>> from datetime import datetime
>>> from dateutil.relativedelta import relativedelta
>>> from dateutil.rrule import *
>>> d = datetime.now()
>>> print(d)
2012-12-23 16:31:52.718111

>>> # Next Friday
>>> print(d + relativedelta(weekday=FR))
2012-12-28 16:31:52.718111
>>>

>>> # Last Friday
>>> print(d + relativedelta(weekday=FR(-1)))
2012-12-21 16:31:52.718111
>>>
```

3.14. Finding the Date Range for the Current Month

Problem

You have some code that needs to loop over each date in the current month, and want an efficient way to calculate that date range.

Solution

Looping over the dates doesn't require building a list of all the dates ahead of time. You can just calculate the starting and stopping date in the range, then use `datetime.time` `delta` objects to increment the date as you go.

Here's a function that takes any `datetime` object, and returns a tuple containing the first date of the month and the starting date of the next month:

```
from datetime import datetime, date, timedelta
import calendar

def get_month_range(start_date=None):
    if start_date is None:
```

```
        start_date = date.today().replace(day=1)
    _, days_in_month = calendar.monthrange(start_date.year, start_date.month)
    end_date = start_date + timedelta(days=days_in_month)
    return (start_date, end_date)
```

With this in place, it's pretty simple to loop over the date range:

```
>>> a_day = timedelta(days=1)
>>> first_day, last_day = get_month_range()
>>> while first_day < last_day:
...     print(first_day)
...     first_day += a_day
...
2012-08-01
2012-08-02
2012-08-03
2012-08-04
2012-08-05
2012-08-06
2012-08-07
2012-08-08
2012-08-09
#... and so on...
```

Discussion

This recipe works by first calculating a date correponding to the first day of the month. A quick way to do this is to use the `replace()` method of a `date` or `datetime` object to simply set the `days` attribute to 1. One nice thing about the `replace()` method is that it creates the same kind of object that you started with. Thus, if the input was a `date` instance, the result is a `date`. Likewise, if the input was a `datetime` instance, you get a `datetime` instance.

After that, the `calendar.monthrange()` function is used to find out how many days are in the month in question. Any time you need to get basic information about calendars, the `calendar` module can be useful. `monthrange()` is only one such function that returns a tuple containing the day of the week along with the number of days in the month.

Once the number of days in the month is known, the ending date is calculated by adding an appropriate `timedelta` to the starting date. It's subtle, but an important aspect of this recipe is that the ending date is not to be included in the range (it is actually the first day of the next month). This mirrors the behavior of Python's slices and range operations, which also never include the end point.

To loop over the date range, standard math and comparison operators are used. For example, `timedelta` instances can be used to increment the date. The `<` operator is used to check whether a date comes before the ending date.

Ideally, it would be nice to create a function that works like the built-in `range()` function, but for dates. Fortunately, this is extremely easy to implement using a generator:

```
def date_range(start, stop, step):
    while start < stop:
        yield start
        start += step
```

Here is an example of it in use:

```
>>> for d in date_range(datetime(2012, 9, 1), datetime(2012,10,1),
                        timedelta(hours=6)):
...     print(d)
...
2012-09-01 00:00:00
2012-09-01 06:00:00
2012-09-01 12:00:00
2012-09-01 18:00:00
2012-09-02 00:00:00
2012-09-02 06:00:00
...
>>>
```

Again, a major part of the ease of implementation is that dates and times can be manipulated using standard math and comparison operators.

3.15. Converting Strings into Datetimes

Problem

Your application receives temporal data in string format, but you want to convert those strings into `datetime` objects in order to perform nonstring operations on them.

Solution

Python's standard `datetime` module is typically the easy solution for this. For example:

```
>>> from datetime import datetime
>>> text = '2012-09-20'
>>> y = datetime.strptime(text, '%Y-%m-%d')
>>> z = datetime.now()
>>> diff = z - y
>>> diff
datetime.timedelta(3, 77824, 177393)
>>>
```

Discussion

The `datetime.strptime()` method supports a host of formatting codes, like `%Y` for the four-digit year and `%m` for the two-digit month. It's also worth noting that these format-

ting placeholders also work in reverse, in case you need to represent a `datetime` object in string output and make it look nice.

For example, let's say you have some code that generates a datetime object, but you need to format a nice, human-readable date to put in the header of an auto-generated letter or report:

```
>>> z
datetime.datetime(2012, 9, 23, 21, 37, 4, 177393)
>>> nice_z = datetime.strftime(z, '%A %B %d, %Y')
>>> nice_z
'Sunday September 23, 2012'
>>>
```

It's worth noting that the performance of `strptime()` is often much worse than you might expect, due to the fact that it's written in pure Python and it has to deal with all sorts of system locale settings. If you are parsing a lot of dates in your code and you know the precise format, you will probably get much better performance by cooking up a custom solution instead. For example, if you knew that the dates were of the form "YYYY-MM-DD," you could write a function like this:

```
from datetime import datetime
def parse_ymd(s):
    year_s, mon_s, day_s = s.split('-')
    return datetime(int(year_s), int(mon_s), int(day_s))
```

When tested, this function runs over seven times faster than `datetime.strptime()`. This is probably something to consider if you're processing large amounts of data involving dates.

3.16. Manipulating Dates Involving Time Zones

Problem

You had a conference call scheduled for December 21, 2012, at 9:30 a.m. in Chicago. At what local time did your friend in Bangalore, India, have to show up to attend?

Solution

For almost any problem involving time zones, you should use the `pytz` module. This package provides the Olson time zone database, which is the de facto standard for time zone information found in many languages and operating systems.

A major use of `pytz` is in localizing simple dates created with the `datetime` library. For example, here is how you would represent a date in Chicago time:

```
>>> from datetime import datetime
>>> from pytz import timezone
```

```
>>> d = datetime(2012, 12, 21, 9, 30, 0)
>>> print(d)
2012-12-21 09:30:00
>>>

>>> # Localize the date for Chicago
>>> central = timezone('US/Central')
>>> loc_d = central.localize(d)
>>> print(loc_d)
2012-12-21 09:30:00-06:00
>>>
```

Once the date has been localized, it can be converted to other time zones. To find the same time in Bangalore, you would do this:

```
>>> # Convert to Bangalore time
>>> bang_d = loc_d.astimezone(timezone('Asia/Kolkata'))
>>> print(bang_d)
2012-12-21 21:00:00+05:30
>>>
```

If you are going to perform arithmetic with localized dates, you need to be particularly aware of daylight saving transitions and other details. For example, in 2013, U.S. standard daylight saving time started on March 13, 2:00 a.m. local time (at which point, time skipped ahead one hour). If you're performing naive arithmetic, you'll get it wrong. For example:

```
>>> d = datetime(2013, 3, 10, 1, 45)
>>> loc_d = central.localize(d)
>>> print(loc_d)
2013-03-10 01:45:00-06:00
>>> later = loc_d + timedelta(minutes=30)
>>> print(later)
2013-03-10 02:15:00-06:00       # WRONG! WRONG!
>>>
```

The answer is wrong because it doesn't account for the one-hour skip in the local time. To fix this, use the `normalize()` method of the time zone. For example:

```
>>> from datetime import timedelta
>>> later = central.normalize(loc_d + timedelta(minutes=30))
>>> print(later)
2013-03-10 03:15:00-05:00
>>>
```

Discussion

To keep your head from completely exploding, a common strategy for localized date handling is to convert all dates to UTC time and to use that for all internal storage and manipulation. For example:

```
>>> print(loc_d)
2013-03-10 01:45:00-06:00
>>> utc_d = loc_d.astimezone(pytz.utc)
>>> print(utc_d)
2013-03-10 07:45:00+00:00
>>>
```

Once in UTC, you don't have to worry about issues related to daylight saving time and other matters. Thus, you can simply perform normal date arithmetic as before. Should you want to output the date in localized time, just convert it to the appropriate time zone afterward. For example:

```
>>> later_utc = utc_d + timedelta(minutes=30)
>>> print(later_utc.astimezone(central))
2013-03-10 03:15:00-05:00
>>>
```

One issue in working with time zones is simply figuring out what time zone names to use. For example, in this recipe, how was it known that "Asia/Kolkata" was the correct time zone name for India? To find out, you can consult the `pytz.country_timezones` dictionary using the ISO 3166 country code as a key. For example:

```
>>> pytz.country_timezones['IN']
['Asia/Kolkata']
>>>
```



By the time you read this, it's possible that the `pytz` module will be deprecated in favor of improved time zone support, as described in PEP 431. Many of the same issues will still apply, however (e.g., advice using UTC dates, etc.).